Conversational agent

In [24]:
```
import os
import openai

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']
```

In [25]:
```
from langchain.tools import tool
```

In [26]:
```
import requests
from pydantic import BaseModel, Field
import datetime

# Define the input schema
class OpenMeteoInput(BaseModel):
    latitude: float = Field(..., description="Latitude of the location to fe
    longitude: float = Field(..., description="Longitude of the location to

@tool(args_schema=OpenMeteoInput)
def get_current_temperature(latitude: float, longitude: float) -> dict:
    """Fetch current temperature for given coordinates."""

    BASE_URL = "https://api.open-meteo.com/v1/forecast"

    # Parameters for the request
    params = {
        'latitude': latitude,
        'longitude': longitude,
        'hourly': 'temperature_2m',
        'forecast_days': 1,
    }

    # Make the request
    response = requests.get(BASE_URL, params=params)

    if response.status_code == 200:
        results = response.json()
    else:
        raise Exception(f"API Request failed with status code: {response.sta

    current_utc_time = datetime.datetime.utcnow()
    time_list = [datetime.datetime.fromisoformat(time_str.replace('Z', '+00:
    temperature_list = results['hourly']['temperature_2m']

    closest_time_index = min(range(len(time_list)), key=lambda i: abs(time_l
    current_temperature = temperature_list[closest_time_index]

    return f'The current temperature is {current_temperature}°C'
```

In [27]:
```
import wikipedia

@tool
def search_wikipedia(query: str) -> str:
    """Run Wikipedia search and get page summaries."""
    page_titles = wikipedia.search(query)
    summaries = []
    for page_title in page_titles[: 3]:
        try:
            wiki_page =  wikipedia.page(title=page_title, auto_suggest=False
            summaries.append(f"Page: {page_title}\nSummary: {wiki_page.summa
        except (
            self.wiki_client.exceptions.PageError,
            self.wiki_client.exceptions.DisambiguationError,
        ):
            pass
    if not summaries:
        return "No good Wikipedia Search Result was found"
    return "\n\n".join(summaries)
```

In [28]:
```
tools = [get_current_temperature, search_wikipedia]
```

In [29]:
```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.tools.render import format_tool_to_openai_function
from langchain.agents.output_parsers import OpenAIFunctionsAgentOutputParser
```

In [30]:
```
functions = [format_tool_to_openai_function(f) for f in tools]
model = ChatOpenAI(temperature=0).bind(functions=functions)
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful but sassy assistant"),
    ("user", "{input}"),
])
chain = prompt | model | OpenAIFunctionsAgentOutputParser()
```

In [31]:
```
result = chain.invoke({"input": "what is the weather is sf?"})
```

In [32]:
```
result.tool
```

Out[32]:
```
'get_current_temperature'
```

In [33]:
```
result.tool_input
```

Out[33]:
```
{'latitude': 37.7749, 'longitude': -122.4194}
```

In [37]:
```
from langchain.prompts import MessagesPlaceholder
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful but sassy assistant"),
    ("user", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])
```

In [38]:
```
chain = prompt | model | OpenAIFunctionsAgentOutputParser()
```

In [39]:
```
result1 = chain.invoke({
    "input": "what is the weather is sf?",
    "agent_scratchpad": []
})
```

In [40]:
```
result1.tool
```

Out[40]:
```
'get_current_temperature'
```

In [41]:
```
observation = get_current_temperature(result1.tool_input)
```

In [42]:
```
observation
```

Out[42]:
```
'The current temperature is 6.4°C'
```

In [43]:
```
type(result1)
```

Out[43]:
```
langchain.schema.agent.AgentActionMessageLog
```

In [44]:
```
from langchain.agents.format_scratchpad import format_to_openai_functions
```

In [45]:
```
result1.message_log
```

Out[45]:
```
[AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_cu
rrent_temperature', 'arguments': '{"latitude":37.7749,"longitude":-122.419
4}'}})]
```

In [46]:
```
format_to_openai_functions([(result1, observation), ])
```

Out[46]:
```
[AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_cu
rrent_temperature', 'arguments': '{"latitude":37.7749,"longitude":-122.419
4}'}}),
 FunctionMessage(content='The current temperature is 6.4°C', name='get_curr
ent_temperature')]
```

In [47]:
```
result2 = chain.invoke({
    "input": "what is the weather is sf?",
    "agent_scratchpad": format_to_openai_functions([(result1, observation)])
})
```

In [48]:
```
result2
```

Out[48]:
```
AgentFinish(return_values={'output': 'The current temperature in San Franci
sco is 6.4°C.'}, log='The current temperature in San Francisco is 6.4°C.')
```

In [49]:
```
from langchain.schema.agent import AgentFinish
def run_agent(user_input):
    intermediate_steps = []
    while True:
        result = chain.invoke({
            "input": user_input,
```

```
            "agent_scratchpad": format_to_openai_functions(intermediate_step
        })
        if isinstance(result, AgentFinish):
            return result
        tool = {
            "search_wikipedia": search_wikipedia,
            "get_current_temperature": get_current_temperature,
        }[result.tool]
        observation = tool.run(result.tool_input)
        intermediate_steps.append((result, observation))
```

In [50]:
```
from langchain.schema.runnable import RunnablePassthrough
agent_chain = RunnablePassthrough.assign(
    agent_scratchpad= lambda x: format_to_openai_functions(x["intermediate_s
) | chain
```

In [51]:
```
def run_agent(user_input):
    intermediate_steps = []
    while True:
        result = agent_chain.invoke({
            "input": user_input,
            "intermediate_steps": intermediate_steps
        })
        if isinstance(result, AgentFinish):
            return result
        tool = {
            "search_wikipedia": search_wikipedia,
            "get_current_temperature": get_current_temperature,
        }[result.tool]
        observation = tool.run(result.tool_input)
        intermediate_steps.append((result, observation))
```

In [52]:
```
run_agent("what is the weather is sf?")
```

Out[52]:
```
AgentFinish(return_values={'output': 'The current temperature in San Franci
sco is 6.4°C.'}, log='The current temperature in San Francisco is 6.4°C.')
```

In [53]:
```
run_agent("what is langchain?")
```

Out[53]:
```
AgentFinish(return_values={'output': 'LangChain is a framework designed to
simplify the creation of applications using large language models (LLMs). I
t is a language model integration framework that can be used for document a
nalysis and summarization, chatbots, and code analysis.'}, log='LangChain i
s a framework designed to simplify the creation of applications using large
language models (LLMs). It is a language model integration framework that c
an be used for document analysis and summarization, chatbots, and code anal
ysis.')
```

In [54]:
```
run_agent("hi!")
```

Out[54]:
```
AgentFinish(return_values={'output': 'Hello! How can I assist you today?'},
log='Hello! How can I assist you today?')
```

In [55]:
```
from langchain.agents import AgentExecutor
agent_executor = AgentExecutor(agent=agent_chain, tools=tools, verbose=True)
```

In [56]:
```
agent_executor.invoke({"input": "what is langchain?"})
```

```
> Entering new AgentExecutor chain...

Invoking: `search_wikipedia` with `{'query': 'Langchain'}`


Page: LangChain
Summary: LangChain is a framework designed to simplify the creation of appli
cations using large language models (LLMs). As a language model integration
framework, LangChain's use-cases largely overlap with those of language mode
ls in general, including document analysis and summarization, chatbots, and
code analysis.

Page: OpenAI
Summary: OpenAI is a U.S. based artificial intelligence (AI) research organi
zation founded in December 2015, researching artificial intelligence with th
e goal of developing "safe and beneficial" artificial general intelligence,
which it defines as "highly autonomous systems that outperform humans at mos
t economically valuable work".
As one of the leading organizations of the AI spring, it has developed sever
al large language models, advanced image generation models, and previously,
released open-source models. Its release of ChatGPT has been credited with s
tarting the AI spring.The organization consists of the non-profit OpenAI, In
c. registered in Delaware and its for-profit subsidiary OpenAI Global, LLC.
It was founded by Ilya Sutskever, Greg Brockman, Trevor Blackwell, Vicki Che
ung, Andrej Karpathy, Durk Kingma, Jessica Livingston, John Schulman, Pamela
Vagata, and Wojciech Zaremba, with Sam Altman and Elon Musk serving as the i
nitial board members. Microsoft provided OpenAI Global LLC with a $1 billion
investment in 2019 and a $10 billion investment in 2023, with a significant
portion of the investment in the form of computational resources on Microsof
t's Azure cloud service.On November 17, 2023, the board removed Altman as CE
O, while Brockman was removed as chairman and then resigned as president. Fo
ur days later, both returned after negotiations with the board, and most of
the board members resigned. The new initial board included former Salesforce
co-CEO Bret Taylor as chairman. It was also announced that Microsoft will ha
ve a non-voting board seat.

Page: DataStax
Summary: DataStax, Inc. is a real-time data for AI company based in Santa Cl
ara, California. Its product Astra DB is a cloud database-as-a-service based
on Apache Cassandra. DataStax also offers DataStax Enterprise (DSE), an on-p
remises database built on Apache Cassandra, and Astra Streaming, a messaging
and event streaming cloud service based on Apache Pulsar. As of June 2022, t
he company has roughly 800 customers distributed in over 50 countries.LangCh
ain is a framework designed to simplify the creation of applications using l
arge language models (LLMs). It is a language model integration framework th
at can be used for document analysis and summarization, chatbots, and code a
nalysis.

> Finished chain.
```

Out[56]:
```
{'input': 'what is langchain?',
 'output': 'LangChain is a framework designed to simplify the creation of a
pplications using large language models (LLMs). It is a language model inte
gration framework that can be used for document analysis and summarization,
chatbots, and code analysis.'}
```

In [57]:
```
agent_executor.invoke({"input": "my name is bob"})
```

```
> Entering new AgentExecutor chain...
Nice to meet you, Bob! How can I assist you today?

> Finished chain.
```

Out[57]:
```
{'input': 'my name is bob',
 'output': 'Nice to meet you, Bob! How can I assist you today?'}
```

In [58]:
```
agent_executor.invoke({"input": "what is my name"})
```

```
> Entering new AgentExecutor chain...
I'm sorry, I don't have access to your personal information. How can I assis
t you today?

> Finished chain.
```

Out[58]:
```
{'input': 'what is my name',
 'output': "I'm sorry, I don't have access to your personal information. Ho
w can I assist you today?"}
```

In [59]:
```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful but sassy assistant"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("user", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])
```

In [60]:
```
agent_chain = RunnablePassthrough.assign(
    agent_scratchpad= lambda x: format_to_openai_functions(x["intermediate_s
) | prompt | model | OpenAIFunctionsAgentOutputParser()
```

In [61]:
```
from langchain.memory import ConversationBufferMemory
memory = ConversationBufferMemory(return_messages=True,memory_key="chat_hist
```

In [62]:
```
agent_executor = AgentExecutor(agent=agent_chain, tools=tools, verbose=True,
```

In [63]:
```
agent_executor.invoke({"input": "my name is bob"})
```

```
> Entering new AgentExecutor chain...
Hello Bob! How can I assist you today?

> Finished chain.
```

Out[63]:
```
{'input': 'my name is bob',
 'chat_history': [HumanMessage(content='my name is bob'),
  AIMessage(content='Hello Bob! How can I assist you today?')],
 'output': 'Hello Bob! How can I assist you today?'}
```

In [64]:
```
agent_executor.invoke({"input": "whats my name"})
```

```
> Entering new AgentExecutor chain...
Your name is Bob. How can I assist you today, Bob?

> Finished chain.
```

Out[64]:
```
{'input': 'whats my name',
 'chat_history': [HumanMessage(content='my name is bob'),
  AIMessage(content='Hello Bob! How can I assist you today?'),
  HumanMessage(content='whats my name'),
  AIMessage(content='Your name is Bob. How can I assist you today, Bob?')],
 'output': 'Your name is Bob. How can I assist you today, Bob?'}
```

In [65]:
```
agent_executor.invoke({"input": "whats the weather in sf?"})
```

```
> Entering new AgentExecutor chain...

Invoking: `get_current_temperature` with `{'latitude': 37.7749, 'longitude':
-122.4194}`


The current temperature is 6.4°CThe current temperature in San Francisco is
6.4°C. Is there anything else you would like to know?

> Finished chain.
```

Out[65]:
```
{'input': 'whats the weather in sf?',
 'chat_history': [HumanMessage(content='my name is bob'),
  AIMessage(content='Hello Bob! How can I assist you today?'),
  HumanMessage(content='whats my name'),
  AIMessage(content='Your name is Bob. How can I assist you today, Bob?'),
  HumanMessage(content='whats the weather in sf?'),
  AIMessage(content='The current temperature in San Francisco is 6.4°C. Is
there anything else you would like to know?')],
 'output': 'The current temperature in San Francisco is 6.4°C. Is there any
thing else you would like to know?'}
```

Create a chatbot

In [66]:
```
@tool
def create_your_own(query: str) -> str:
    """This function can do whatever you would like once you fill it in """
    print(type(query))
    return query[::-1]
```

In [67]:
```
tools = [get_current_temperature, search_wikipedia, create_your_own]
```

In [68]:
```
import panel as pn  # GUI
pn.extension()
import panel as pn
import param

class cbfs(param.Parameterized):

    def __init__(self, tools, **params):
        super(cbfs, self).__init__( **params)
        self.panels = []
        self.functions = [format_tool_to_openai_function(f) for f in tools]
        self.model = ChatOpenAI(temperature=0).bind(functions=self.functions
        self.memory = ConversationBufferMemory(return_messages=True,memory_k
        self.prompt = ChatPromptTemplate.from_messages([
            ("system", "You are helpful but sassy assistant"),
            MessagesPlaceholder(variable_name="chat_history"),
            ("user", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad")
        ])
        self.chain = RunnablePassthrough.assign(
            agent_scratchpad = lambda x: format_to_openai_functions(x["inter
        ) | self.prompt | self.model | OpenAIFunctionsAgentOutputParser()
        self.qa = AgentExecutor(agent=self.chain, tools=tools, verbose=False

    def convchain(self, query):
        if not query:
            return
        inp.value = ''
        result = self.qa.invoke({"input": query})
        self.answer = result['output']
        self.panels.extend([
            pn.Row('User:', pn.pane.Markdown(query, width=450)),
            pn.Row('ChatBot:', pn.pane.Markdown(self.answer, width=450, styl
        ])
        return pn.WidgetBox(*self.panels, scroll=True)


    def clr_history(self,count=0):
        self.chat_history = []
        return
```

In [69]:
```
cb = cbfs(tools)

inp = pn.widgets.TextInput( placeholder='Enter text here…')

conversation = pn.bind(cb.convchain, inp)

tab1 = pn.Column(
    pn.Row(inp),
    pn.layout.Divider(),
    pn.panel(conversation,  loading_indicator=True, height=400),
    pn.layout.Divider(),
)

dashboard = pn.Column(
```

```
    pn.Row(pn.pane.Markdown('# QnA_Bot')),
    pn.Tabs(('Conversation', tab1))
)
dashboard
```

Out[69]:

QnA_Bot

Conversation

Enter text here...

In [ ]:

In [ ]: